

MATH 14 Slides — Authoring Guide & Lessons Learned

PreTeXt reveal.js slideshow deck. This file records how the project is built and the specific gotchas already solved, so we don't rediscover them. Read it before editing slides.

1. Project layout & build

```
source/main.ptx # the slideshow source (all slides) project.ptx # target "slides", format
revealjs publication/publication.ptx # directories + revealjs appearance (theme, custom-css
assets/ # external files: custom.css, *.ggb (external dir) generated-assets/ # PreFigure/la
SVGs (+ a committed qrcode stub) .devcontainer/ # dev container (needs Docker running)
output/slides/main.html # built deck
```

Building

- Build inside the dev container via PreTeXt Tools Nj "Build default target" (status-bar "PreTeXt" button Nj menu). Output goes to output/slides/.
- The message PTX:ERROR: Table of Contents level ... not determined during asset generation is harmless for slideshows — the build still says Success! Built requested target(s) without errors.
- pretext generate ... exits non-zero on that TOC message even when assets are fine; the subsequent pretext build is the real signal. CI uses pretext generate ... || true.
- Builds sometimes silently no-op. After a build, confirm output/slides/main.html mtime actually changed before trusting the result.

Dev container (.devcontainer/post-create.sh) must install

- texlive-latex-extra texlive-pictures texlive-fonts-recommended **texlive-xetex** (PreTeXt compiles latex-image/TikZ with xelatex, not pdflatex — missing this was the first build failure), pdf2svg, libcairo2-dev, nodejs npm.
 - pip install pretext prefig, then prefig init (downloads PreFigure's MathJax engine; without it PreFigure diagrams fail).
 - Requires Docker Desktop installed & running, plus the Dev Containers extension.
-

2. Reveal.js slide structure — the #1 rule

Reveal clips anything taller than one slide. Internal slide area is only ~960×530.

- `<section>` Nj horizontal slide (also a section-title slide); each `<slide>` Nj vertical sub-slide. Navigate as `#/<section>/<slide>` (e.g. `#/2/6`).
 - One idea per slide. A theorem + its figure, or a long example + its figure, will overflow and the figure/last lines get cut off. Put each figure on its own slide.
 - Body font size is set small (`--r-main-font-size: 27px` in `custom.css`) specifically so a full example (statement + parts A–C) fits on one slide. Don't bump it up blindly.
-

3. Figures — which technology to use

| Figure kind | Use | Notes | |---|---|---| | 2D diagrams (trees, plane geometry) | PreFigure | `<prefigure label="...">` — the `label` attr is required for a stable asset filename. | | 3D static (helix, surfaces) | TikZ / pgfplots `<latex-image>` | Compiles to SVG via xelatex. Reliable, crisp. `latex-image-preamble` needs `\usepackage{pgfplots}` + `\usetikzlibrary{arrows.meta}`. | | Interactive | GeoGebra (see §4) | JSXGraph does NOT work — see below. |

Do NOT use JSXGraph slates in reveal

PreTeXt's revealjs target renders a `<slate surface="jsxgraph">`'s code as visible text with smart-quotes (' Nj ') instead of running it, and heavy 3D boards hang the page. Use TikZ (static) or GeoGebra (interactive) instead.

4. GeoGebra embedding — playbook (this ate the most time)

4a. What works: inline base64 in a geogebra slate

The reliable way to embed a specific construction: `xml <interactive xml:id="myfig" platform="geogebra" width="55%" aspect="1:1"> <slate xml:id="myfig-slate" surface="geogebra" base64="UESDBB... .ggb zip, base64"> setCoordSystem(-3.5,3.5,-3.5,3.5) <!-- optional API commands, one per line --> </slate> </interactive>` - The base64 is the base64 of the .ggb (a ZIP starting PK/UESD). Get it from a GeoGebra activity's offline HTML export: each applet appears as `ggbBase64": "..."`, in figure order (block N = "Figure (N)"). - Slate text lines are GeoGebra API calls without the `ggbApplet.` prefix (PreTeXt adds it). Straight quotes ARE preserved for geogebra slates (unlike `jsxgraph`). Useful ones: `setWidth(px)`, `setHeight(px)`, `setCoordSystem(xmin,xmax,ymin,ymax)`, `evalCommand("...")`, `setVisibleInView(obj,1,true)` (as `evalCommand`). - Sizing: `setWidth/setHeight` set the applet's own pixel size; pair with CSS on `iframe#<xml:id>` so the applet fills the frame (CSS alone letterboxes it).

4b. What does NOT work

- `<interactive geogebra="ID">` needs a PUBLIC material ID. Individual figures inside someone's activity are private (`geogebra.org/m/<id>` Nj 404) and can't be embedded by ID. Only the whole activity is public.
- `.ggb` file via `@source/filename`: GeoGebra's player resolves the relative `filename` against the CDN, not the page, so the applet loads empty (0 objects). Use `base64` instead.
- Two-pane worksheet layouts don't reproduce. If a construction has a second graphics view / side panel (e.g. a color legend on white), the embedded player ignores the pane divider and collapses the panel to a sliver. Widening the whole applet just makes it render more field arrows and freezes the browser. Don't chase this — instead:
 - render the main graphics view cleanly (`setCoordSystem` to frame it), and
 - recreate the panel text as `PreTeXt` slide content next to the figure (see §5c).
- Objects assigned to a 2nd view (`ev bitmask`) won't show in view 1. Forcing them with `setVisibleInView` makes them overlap the graph. Prefer slide text.

4c. Performance warning

Dense vector-field / heavy 3D GeoGebra applets take ~60–75 s to paint and freeze the CDP/preview browser while doing so. Keep applets modest; expect slow previews.

5. Styling & CSS gotchas

5a. Theme + custom CSS (publication file)

```
xml <revealjs> <appearance theme="simple" custom-css="external/custom.css"/> </revealjs>
```

- custom-css value must be `external/custom.css` — `PreTeXt` copies files from the external dir (assets/) to output/slides/external/ but writes the href verbatim. Just `custom.css` gives a 404.

- Current look: Times New Roman, GeoGebra-indigo headings (#5b57d1), blue/red accents, clean white. Callout boxes are tinted with a colored left rule.

5b. MathJax is v2.7.8 here

- `\textcolor{...}` does NOT render (fails silently, leaving blank math). To color text/math, set CSS `color` on the container — MathJax inherits `currentColor`.

5c. `xml:id` on a `<p>` does NOT become an HTML `id`

So you can't target a paragraph by `#its-id`. Scope by structure instead. Example used for the Figure 2 color legend (first three `<p>` after the figure, colored to match the graph): `css .reveal section:has(iframe#fig2-work-paths) > div[align="left"] > p:nth-of-type(1), .reveal section > div[align="left"] > p:nth-of-type(1) * { color:#cc0000 !important; } /* ...:nth-of-type(`

green #009900, :nth-of-type(3) black ... */ section:has(iframe#...) scopes to the right slide; > div[align="left"] > p:nth-of-type(n) picks the paragraphs (the interactive/opener are divs, so the legend <p>s are the first ones). :has() is fine — the deck renders in modern Chrome.

5d. GeoGebra iframe sizing

The applet iframe id equals the interactive's xml:id. Example: `css .reveal iframe#fig2-work-paths { width: 44vh !important; height: 44vh !important; max-width: 90vw; margin: 0 auto !important; display: block; border: none; }` A square frame keeps a square GeoGebra view undistorted (round circles stay round). A wide frame with a square view left-aligns the drawing; a wide view stretches circles.

6. Verifying the deck (preview)

- Start the server via PreTeXt Tools "View full document"; it serves at `http://localhost:<port>/output`. The port changes on rebuild (8129Nj8130Nj8133...) — check the PreTeXt Tools output for the current one.
 - The preview browser caches `main.html` and `external/custom.css` aggressively. To see changes: load a fresh `?cachebust=N` query, and if CSS is stale, rewrite the stylesheet link href with a query via JS, or load from a fresh server port (new port = fresh cache).
 - Screenshots of slides with heavy GeoGebra applets time out until the applet finishes (~60–75 s). Wait, then screenshot. Chrome may surface an unrelated tab — screenshot the specific `tabId` via the browser MCP rather than the desktop.
-

7. Editing source/main.ptx — practical notes

- The file is large because embedded GeoGebra base64 strings are ~90 KB each. The Read tool may refuse it; use `offset/limit`, `grep`, or a python script.
 - To edit around a huge base64 slate, use a python script (match by a short unique neighbor like the `setCoordSystem(...)` line), not the whole-string Edit tool.
 - When inserting after an element, remember `str.find("</interactive>")` returns the first occurrence. There are multiple interactives — scope by locating `xml:id="the-one-you-want"` first, then the next `</interactive>`.
-

8. Quick "known-good" recipes

- New content slide: `<slide><title>...</title> ...one theorem/example... </slide>`. If it has a figure, give the figure its own following `<slide>`.
- Embed an existing GeoGebra figure: get its base64 from the activity's HTML export Nj `<slate surface="geogebra" base64="..."> setCoordSystem(...) </slate>` Nj size via CSS on `iframe#<xml`
- Colored inline text/legend: `plain <p> + scoped section:has(...) > div > p:nth-of-type(n)` CSS color (NOT `\textcolor`).